International Journal of Software Engineering
and Knowledge Engineering
Vol. 25, No. 6 (2015) 1017–1032
© World Scientific Publishing Company
DOI: 10.1142/S0218194015500151

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

Probabilistic Approach to Predicting Risk in Software Projects Using Software Repository Data

Changkyun Jeon*, Neunghoe Kim† and Hoh Peter In‡

Department of Computer Science and Engineering
Korea University, Seoul, Korea
**curt_jeon@korea.ac.kr*
†nunghoi@korea.ac.kr
‡hoh_in@korea.ac.kr

Received 28 May 2014
Revised 29 July 2014
Accepted 22 September 2014

Although the factors that need to be focused on for a successful software project appear to be difficult to define, risk management has become one of the key activities for achieving such success because significant risk is involved in each software development phase. Software project failures are often a result of insufficient and ineffective risk information regarding the future. To overcome this, software risk prediction should be performed in advance to allow project managers insight into providing more valuable information for decision making, such as scope coverage, resource allocation, and schedule changes. In this research, we propose a risk prediction model from the perspective of quality using a software repository. We evaluated the risk threat level by mapping some defect attributes that exist in the defect lifecycle, defined their risk threat transition states, and applied a Markov chain for predicting the potential risk level. We evaluated the proposed approach using practical real-industry mobile software projects. The experimental results confirm that our approach is applicable to software threat risk estimation.

Keywords: Software risk; project management; software repository; software engineering.

1. Introduction

The global software market has been forecasted to reach US$396.7 billion in value by 2016, an increase of 35.4% from 2011 [1]. However, software projects are high-risk and challenging in terms of achieving their original goal (on time, on budget, and with the required features and functions). The 2013 CHAOS results show that 39% of all projects succeed, 43% are challenging, and 18% fail [2].

Software development management is mandatory to the success of a software project. A software project can be defined as a series of technical and managerial

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

activities that should meet the terms and conditions listed in the project agreement [3]. Software project management involves iterative and complicated activities based on previous experiences and certain consecutive artifacts. For a software project to be deemed a success, it usually means that the project can be completed within the budget and given timeframe, as well as meet the customer's demand for high-quality performance.

Many previous researches have noted that the most important activity for the success of a software project in the field of software engineering is risk management, followed by technical issues. For example, the Standish Group's report summarized the ingredients for a successful software project, most of which are non-technical factors [4]. Risk management, which is critical to a project management, is one of the nine knowledge areas defined in the Project Management Body of Knowledge [5], and is one of the 25 key process areas defined in Capability Maturity Model Integration (CMMI) [6]. Studies have shown that the presence of risk has led to cancellation of approximately 15% to 35% of software development before the initial objectives were met [7].

Most researches on the risk management of a software project have been focused on the discovery of correlations between risk factors and project outcomes [8, 9]. Although they have explored different types of risks influencing both the process and product outcomes in a software development project, software project practitioners have had trouble determining when the allocation of time and resources should be prepared against the real and direct risks to be avoided and mitigated. Resources should be adjusted or reallocated among several projects to enhance the probability of a project's success. However, it may be impossible to manage several projects at the same time and proactively share the resources among them without predicting their risk. Thus, risk prediction in a software project is mandatory information for compromising project decisions.

In software engineering, risk management can be considered as every single quality assurance activity for dealing with three constraints, i.e. scope, cost, and schedule, of the project management. In addition, it can also be stated that certain attributes (such as the *severity* of a reported defect and the *frequency of occurrence* when carrying out the required functions and features) are dominant factors representing the quality level of the deliverable software. For example, the rare occurrence of a critical defect causing a program to fail or stop in a high-reliability system, such as a nuclear power plant or an air-traffic control system, can lead to a catastrophic disaster. Moreover, minor negligible defects in a mobile device that occur during every user action can result in poor user satisfaction and complaints.

This paper deals with the prediction of software risk based on quality. All defects must be recorded, tracked, and managed until the end of a project using a software repository. The Bugzilla, Git, and Jira defect tracking systems (DTSs) are widely used for such purposes [10]. Thus, it is possible to analyze the historical quality trends in a DTS, use the resulting information to predict the risk in an upcoming

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

software project, and provide insight to a project manager to avoid, mitigate, or plan a contingent response against such risk. Knowing that the probability of a failure is high before a project reaches certain milestones allows the project management to take risk-reduction measures, such as allocating more resources for the development, testing, and inspection of the product, or a delay in its delivery to customers.

This paper offers the following contributions:

- We present and qualitatively analyze software risk using certain defect attributes after an analysis of the repository data from a DTS.
- We can predict the potential risk of a software project at a future time, and provide insight to the project manager to avoid, mitigate, or plan a contingent response as a risk management strategy.

The remainder of this paper is organized as follows. Section 2 briefly introduces some background information on a risk matrix (RM) for scoring the risk level, and on the Markov chain (MC) model. Section 3 describes our proposed model in detail, and shows how to calculate a transition matrix of defect risk and obtain the initial probability for predicting risk during the next time frame. To evaluate our proposed model, a case study and its results are described in Sec. 4. Section 5 summarizes previous related work, and Sec. 6 discusses limitations of our study. Finally, we provide some concluding remarks in Sec. 7, along with directions for future research.

2. Background

2.1. *Risk matrix theory for software quality*

According to the risk matrix theory, the risk assessment, R, of a risk event is determined by risk factors that include the risk influence and risk probability [11]. In software engineering, risk refers to the possibility that a program's requirements (quality) cannot be met through available technology or through suitable engineering procedures or processes with the triple constraints (schedule, budget, and scope) listed above. The risk level of each issue can be obtained according to the risk matrix, and is presented and managed in the domain knowledge.

***Impact* (I):** *Assess the impact that a quality risk can have on a software project depending on severity of the defect* (*Table* 1).
***Probability of Occurrence* (P):** *Assess the probability that a defect will occur* (*Table* 2).
***Risk Rating* (R):** *Determine the risk rating by multiplying each* (I, P) *pair into the default matrix shown in Table* 3 *and in Eq.* (1) *below.*

$$R = I \times P \tag{1}$$

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 1. Risk matrix impact assessments (I).

Impact category	Definition
Critical (C)	An event that, if it occurs, will cause a program failure (inability to achieve the minimum acceptable requirements).
Serious (S)	An event that, if it occurs, will cause major cost/schedule increases. Secondary requirements may not be achieved.
Moderate (Mo)	An event that, if it occurs, will cause moderate cost/schedule increases, but important requirements will still be met.
Minor (Mi)	An event that, if it occurs, will cause small cost/schedule increases, but important requirements will still be achieved.
Negligible (N)	An event that, if it occurs, will have no effect on the program.

Table 2. Probability of occurrence (P).

Probability range (%)	Interpretation
0–10	Very Unlikely to Occur
11–40	Unlikely to Occur
41–60	May Occur About Half of the Time
61–90	Likely to Occur.
90–100	Very Likely to Occur.

Table 3. Example of a risk rating scale (R).

	Negligible	Minor	Moderate	Serious	Critical
0–10%	Low	Low	Low	Medium	Medium
11–40%	Low	Low	Medium	Medium	High
41–60%	Low	Medium	Medium	Medium	High
61–90%	Medium	Medium	Medium	Medium	High
90–100%	Medium	High	High	High	High

We can define the risk in software engineering to identify and describe the defects that might result in a quality issue with regard to the three constraints to a software project.

2.2. *MC model*

An MC [12] model is concerned with a sequence of random variables (i.e. $X_1, X_2, X_3, \ldots$) with the Markov property, namely, that the state in a given time epoch depends only on the state in the previous time epoch, as in Eq. (2) below:

$$Pr(X_{n+1} = x | X_n = x_n, \ldots, X_1 = x_1, X_0 = x_0) = Pr(X_{n+1} = x | X_n = x_n). \quad (2)$$

The possible values of X_i form a countable set, S, which is called the state space of the chain. MCs are often described using a directed graph, where the edges are labeled based on the probability of passing from one state to another (e.g. a finite state machine), as shown in Fig. 1. Here, p_{ij} represents the probability of a transition

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

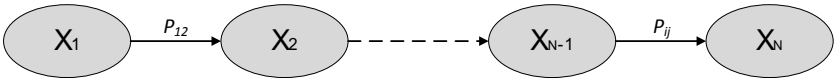

Fig. 1. Overview of MC [11].

from state x_i to state x_j. We can also define the next state at time $t+1$ for the state probability vector v at time t using Eq. (3) and (4) below:

$$P = \begin{pmatrix} p_{11} & p_{21} & \cdots & p_{x-1,1} & p_{x,1} \\ p_{12} & p_{22} & \cdots & p_{x-1,2} & p_{x,2} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ p_{1,x-1} & p_{2,x-1} & \cdots & p_{x-1,x-1} & p_{x,x-1} \\ p_{1,x} & p_{2,x} & \cdots & p_{x,x-1} & p_{x,x} \end{pmatrix} \tag{3}$$

$$v_{t+1} = P \cdot v_t \tag{4}$$

To apply the MC model, state transitions must satisfy the property in which the next state depends only on the current state. In addition, all logs in the software repository must be initiated by a defect reporter based only on the current state, rather than on the previous state. Therefore, the state of each reported defect can be expressed using the Markov property, as normally defined.

3. Approach for Risk Prediction Model (RPM)

In this section, the procedure used to construct the risk prediction model is described in detail. We propose a software RPM that is able to qualify the risk threat of each defect based on the severity and frequency of occurrence, and predict the risk to an upcoming software project. Figure 2 shows the overall structure of the RPM, which is composed of five steps: Risk Threat Rating Mapping, Risk Threat State Definition, Risk Threat State Transition Matrix, Initial Vector for Risk Threat, and Risk Threat Prediction.

In the 'risk threat rating mapping' step, the risk matrix, in which the severity and frequency of occurrence of each defect are mapped into the impact factor, and the probability of the risk matrix are both calculated. This allows the risk matrix table to

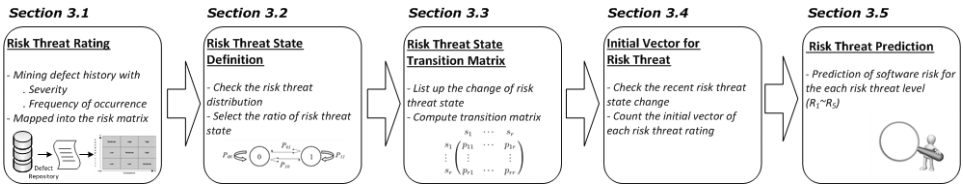

Fig. 2. Procedure for building an RPM.

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

distinguish a specific risk rating from an entire defect list. In the 'risk threat state definition' step, a set of risk states is defined by setting a threshold with regard to the portion of all defects. In the 'risk threat state transition matrix', a transition matrix is calculated among the risk threat states using the defined risk threat states in the previous step. In the 'initial vector for risk threat' step, the initial probability vector for predicting the risk is calculated.

Finally, in the 'risk threat prediction' step, the probability of a defect risk is estimated using the risk threat transition matrix and the initial vector probability calculated in the previous steps. A more detailed description is presented in the following subsections.

3.1. *Risk threat rating mapping* (*Step* 1)

In the Risk Threat Rating Mapping step, there are two tasks performed to map certain attributes in each defect into the impact factor (I) and probability of occurrence (P) of risk matrix. The gathering of the defect history and confirming the severity and frequency of occurrence of each defect are the first tasks. This information is then directly mapped into the impact factor and probability of occurrence in risk matrix.

Similar to the system by commercial and in-house DTS, every defect tracking system has a field showing the severity and frequency of occurrence. Once the tester reports a defect to the defect tracking system, the tester should mark the severity and occurrence rate of the defect based on its urgency and pre-set priorities. The mapping of defects into a risk matrix can be achieved using two metrics. The severity is the extent to which a defect causes a failure that degrades the expected operational capabilities of the system of which the software is a component, and can be classified from a domain-dependent perspective. Table 4 shows the severity definitions in the Bugzillia [9].

Normally, the end users who encounter a failure are the best judges to classify the severity of a defect. However, during the development phases, the tester has the responsibility of judging any existing defects. Another attribute, the frequency of occurrence, is the mean time between failures, as shown in Table 5. This may be a statistical value between any failures, or may be broken down into the rate of

Table 4. Severity description at the Bugzilla [9].

Severity	Description
Blocker	Blocks the development and/or testing work
Critical	Crashes, loss of data, sever memory leaks
Major	Major loss of functionality
Normal	Regular issues, some loss of functionality under specific circumstances
Minor	Minor loss of functionality, or another problem with an easy workaround
Trivial	Cosmetic problem such as misspelled words or misaligned text
Enhancement	Request for enhancement

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

Table 5. Frequency of occurrence.

Frequency	Description
Always	Encountered every time
Likely	Will probably be encountered
Unlikely	Will probably not be encountered

Table 6. Defect risk threat rating matrix.

	Negligible (1)	Minor (2)	Moderate (3)	Serious (4)	Critical (5)
Unlikely (1)	1	2	3	4	5
Likely (2)	2	4	6	8	10
Always (3)	3	6	9	12	15

occurrence between similar failures. It can also be defined as the likelihood of a defect occurring or manifesting itself to the end-user, an organization, a third party, or a service. We have categorized the five severity levels as negligible, minor, moderate, serious, and critical. Further, the frequency of occurrence has three levels, unlikely, likely, and always. This enables the definition of severity and frequency in the various repository data and allows the mapping into a risk matrix for applying to the proposed model.

By multiplying the severity and frequency of occurrence, we can build a defect risk-threat rating matrix, as shown in Table 6. It is a quantitative risk level of addressing and identifying their risk rating comparing with others.

3.2. *Risk threat state definition* (*Step 2*)

Based on the rate mapping results of the risk threat, we need to determine the states of the defect risk threats. To obtain these states, two tasks are performed: First, the distribution of each defect risk threat compared to the entire number of defects per week is needed because the distribution of defect risk threat ratings during each week differs, which also impacts the whole risk level of the software project. For example, in the early stage of development, a higher risk threat rating occupies the majority of reported defects such as a crash or reset owing to the instability of the software. At the end of the development, lower risk threat ratings are frequently found and reported after solving the crucial defects related with blocks in the testing and major losses of functions.

Second, the ratio of each defect risk threat state should be selected, and lastly, the states should be defined. Normally, the number of defects is slightly ramped up to the peak, and then slightly ramped down heading to the end of development.

R, which is a set of defect risk threat ratings, can be defined as follows:

$$R = I \times P, \quad R = \{R_1, R_2, \ldots, R_m\}, \tag{5}$$

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

where I is severity factor, P is frequency of occurrence and m represent the risk threat rating as follows.

$$R_1 : 0-3, \quad R_2 : 4-6, \quad R_3 : 7-9, \quad R_4 : 10-12, \quad R_5 : 13-15 \tag{6}$$

How to define the range of risk threat rating levels depends on the domain. For high reliability in a software project, we need to define whether a higher risk threat state can appropriately hold most of the risk threat rating.

To define the risk threat transition state, S, which is a set of risk rating states, can be defined as follows:

$$S = \{S_1, S_2, \ldots, S_n\}. \tag{7}$$

where n represent the number of states for representing risk threat states.

In this research, we have five states with the same range, which are defined as follows:

$$S_1 : 0-20\%, \quad S_2 : 21-40\%, \quad S_3 : 41-60\%, \quad S_4 : 61-80\%, \quad S_5 : 81-100\% \tag{8}$$

3.3. *Risk threat state transition matrix* (*Step* 3)

During this step, the risk threat transition matrix is calculated. The risk threat states are listed by mapping the transition occurrence data of one risk threat state to another. The matrix is constructed by counting the number of transition from one risk threat states to another. The risk threat state transition probability matrix can be expressed as Eq. (9), satisfying conditional Eq. (10).

$$P_{R_m} = \begin{pmatrix} s_{11} & s_{12} & \cdots & s_{1n} \\ s_{21} & s_{22} & \cdots & s_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ s_{n1} & s_{n2} & \cdots & s_{nn} \end{pmatrix} \tag{9}$$

$$\sum_{j=1}^{n} s_{1j} = 1, \sum_{j=1}^{n} s_{2j} = 1, \ldots, \sum_{j=1}^{n} s_{nj} = 1 \tag{10}$$

where m represents the risk threat rating, and n is the number of states.

In each row, the probabilities of moving from the state represented by that row to the other states are shown. Thus, each row of a transition matrix adds up to 1.

3.4. *Initial vector for risk threat* (*Step* 4)

To obtain the initial probability vector representing the occurrence probability of a risk threat being in the initial state, the recent data on the risk threat state are used, divided into weeks. The initial probability vector is calculated using Eq. (11), satisfying conditional Eq. (12).

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

$$P_{initR_m}(s_1, s_2, \ldots, s_n) = P_{initR_m}\left(\frac{\alpha}{F}, \frac{\beta}{F}, \ldots, \frac{\gamma}{F}\right) \tag{11}$$

$$F = \sum_{i=1}^{n} f_i = \alpha + \beta + \cdots + \gamma \tag{12}$$

where α, β, and γ represent the number of transitions for each threat state s_1, s_2, through to s_n. The initial probability $P_{initRm}(s_i)$ for each state s_i satisfies Eq. (13) because the sum of the initial probabilities must be one.

$$\sum_{i=1}^{n} P_{initR_m}(s_i) = 1 \tag{13}$$

3.5. *Risk threat prediction* (*Step* 5)

The probability of a risk threat occurring in the future is estimated using the transition matrix created in step 3, and the initial vector created in step 4.

$$P(s_1, s_2, \ldots, s_n) = P_{initR_m}(s_1, s_2, \ldots, s_n) \times P_{R_m} \tag{14}$$

where n is the number of states representing a risk rating; m is the specific risk threat rating, such as the highest (5) and lowest (1) risk defects; $P_{initRm}(s_n)$ is the initial probability; and $P_{Rm}(s_n)$ is the state transition probability of a defect risk threat. We can also calculate a specific risk threat state using Eq. (15):

$$P(s_k) = \sum_{i=1}^{n} P(s_i)P_{ik}, \quad 0 < k < n \tag{15}$$

where k is a specific risk rating state such as highest or lowest, n is a number of states. P_{ik} is element of the transition matrix for specific risk threat.

4. Case Study

4.1. *Background*

As a case study, we evaluated the proposed RPM using a previously developed practical mobile software project. This project consists of one platform model with two consecutive products. Defect samples of the platform product are used for building a transition probability matrix and checking the validity of the proposed model. This software project was carried out over a six-month period. The platform product consists of full-featured mobile devices. The first product project was established using value-engineering concepts to reduce the cost value. Some of the software-related components were changed or removed. Accordingly, quite a few of the common parts became variable using the software product-line method. In contrast, the second product had exactly the same common parts as the platform product, but with some features added and user experience design (UXD) concepts

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

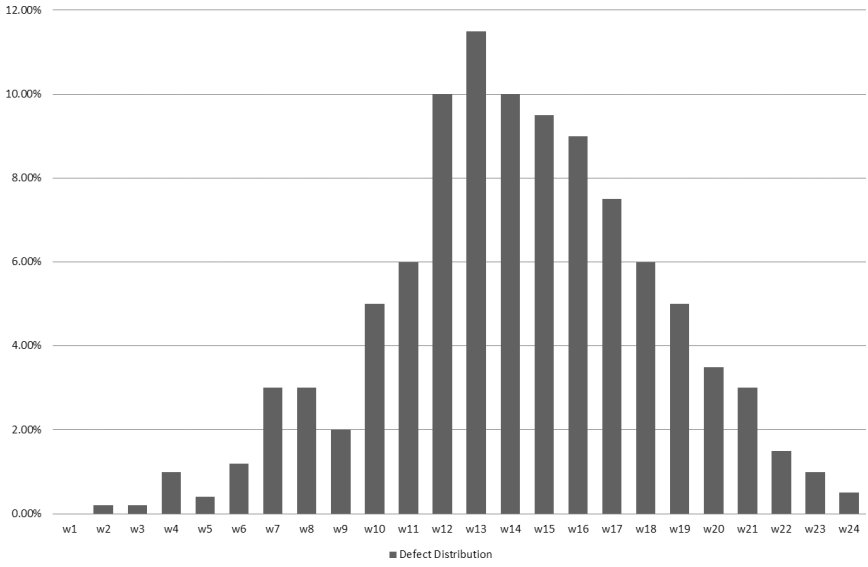

Fig. 3. Actual defect distribution for the case study.

changed. All of the products were developed in this manner by the same development and test team.

Figure 3 shows the actual defect distribution for the series of products through the entire development time. The defect distribution has a normal distribution similar to typical defect trends. For example, the test team took a while to report the defects of the platform product, and further defects remained undiscovered for a certain amount of time owing to the software instability. At some point, the number of defects peaked and then decreased over the remaining periods, approaching zero. Figure 4 shows the actual defect risk threat rating distribution after mapping the severity and frequency of occurrence into the risk matrix for the series of products on a week-by-week basis. In the early phase of development, the higher risk threat (R_5, R_4) had the largest majority of distribution compared with the lower risk threat (R_1, R_2). Conversely, at the end of the development, the majority of the distribution represents lower-risk threats.

A dataset was gathered using an in-house DTS. We used this dataset to build the proposed RPM and validate it by comparing the actual and predicted results. We want to predict the software risk not on a product basis but from the perspective of the entire software project.

4.2. *Building the proposed RPM*

To build an RPM, the risk threat rating is calculated after multiplying the severity and frequency of occurrence at each mined defect from the DTS. We then need to define the ratio of each risk rating to the entire number of risks on a weekly basis. In

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

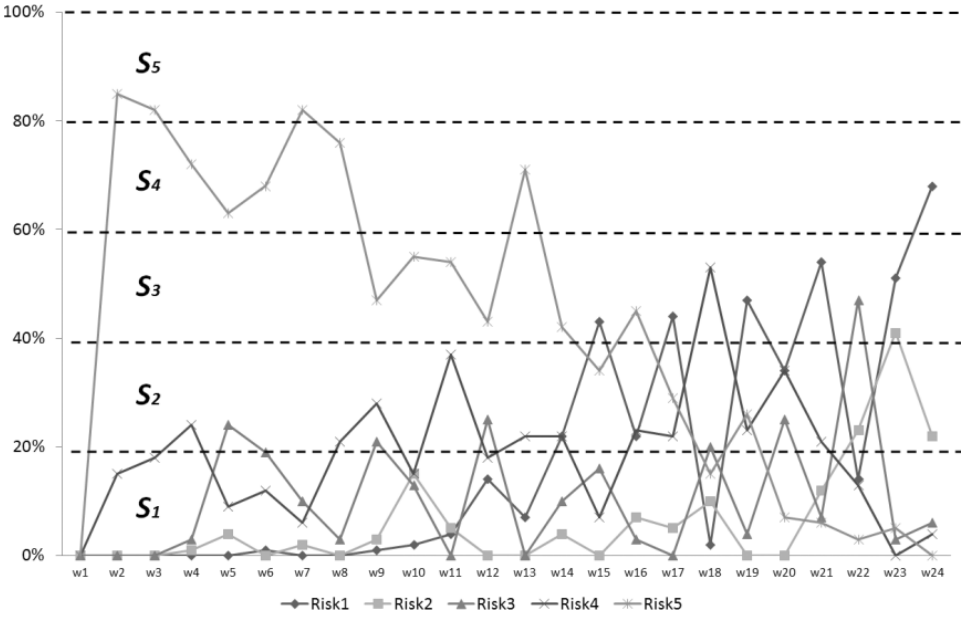

Fig. 4. Actual risk threat rate distribution for the case study.

this case study, we took two risk rates, R_5 (highest risk rating) and R_1 (lowest risk rating), to evaluate our model. As we can see in Fig. 4, the portion of R_5 and R_1 during the second week was around 85% and 0%. In the last week, that value was changed to around 0% and 68%.

To calculate the risk threat rate transition probability matrix, the moving risk states of R_5 and R_1 during the whole development for every week, based on Fig. 5, are listed below.

$\boldsymbol{R_5}$(*Highest Risk Threat*):

$$S_1, S_5, S_5, S_4, S_4, S_4, S_5, S_4, S_3, S_3, S_3, S_3, S_4, S_3, S_2, S_3, S_2, S_1, S_2, S_1, S_1, S_1, S_1, S_1$$

$\boldsymbol{R_1}$(*Lowest Risk Threat*):

$$S_1, S_1, S_1, S_1, S_1, S_1, S_1, S_1, S_1, S_1, S_1, S_1, S_1, S_2, S_3, S_2, S_3, S_1, S_3, S_2, S_3, S_1, S_3, S_4$$

From the above changes in state, the number of transitions from each state (S_1 through S_5) to another state is counted, which can lead to a transition matrix using Eq. (9) and Eq. (10). To apply the MC, the risk state should be actively changed.

$$P_{R_5} = \begin{pmatrix} 4 & 1 & 0 & 0 & 1 \\ 2 & 0 & 1 & 0 & 0 \\ 0 & 2 & 3 & 1 & 0 \\ 0 & 0 & 3 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix} := \begin{pmatrix} 0.67 & 0.17 & 0 & 0 & 0.17 \\ 0.67 & 0 & 0.33 & 0 & 0 \\ 0 & 0.33 & 0.5 & 0.17 & 0 \\ 0 & 0 & 0.6 & 0.2 & 0.2 \\ 0 & 0 & 0 & 0.5 & 0.5 \end{pmatrix} \tag{16}$$

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

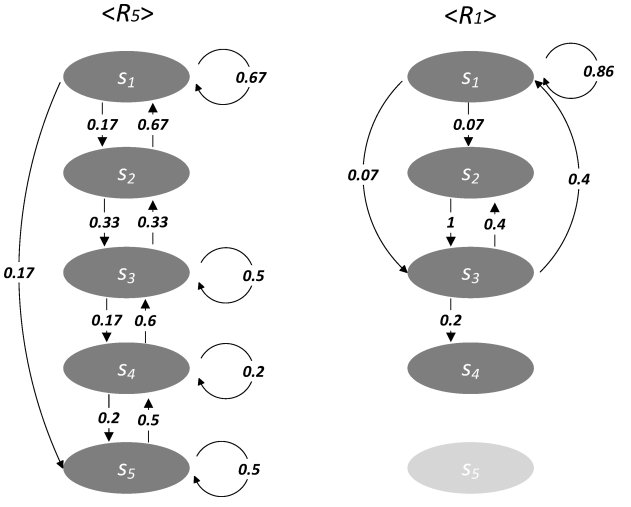

Fig. 5. Risk state diagrams for R_5, R_1.

$$P_{R_1} = \begin{pmatrix} 12 & 1 & 1 & 0 & 0 \\ 0 & 0 & 3 & 0 & 0 \\ 2 & 2 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} := \begin{pmatrix} 0.86 & 0.07 & 0.07 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0.4 & 0.4 & 0 & 0.2 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \tag{17}$$

The entries of the transition matrix must satisfy Eq. (10) such that the rows each add up to one. In addition, the transition matrix for risk ratings R_5 and R_1 can be translated as risk state diagrams for the MC shown in Fig. 5.

To calculate the initial vector for risk threat of R_5 and R_1 in consecutive products, the most-recent risk threat state of a defect for a one-month period is used. The initial vector can be calculated using Eq. (11) and Eq. (12) as follows:

$$\boldsymbol{R_5} : S_5, S_5, S_4, S_3 \rightarrow P_{initR_5}(0, 0, 0.25, 0.25, 0.5) \tag{18}$$

$$\boldsymbol{R_1} : S_1, S_1, S_1, S_1 \rightarrow P_{initR_1}(0.75, 0.25, 0, 0, 0) \tag{19}$$

The probability and portion of each risk threat can be estimated using the transition probability matrix and initial vector through Eq. (14) and Eq. (15). The probabilities of R_5 and R_1 are calculated as follows:

$$\begin{aligned} \text{Probability of } R_5 &= (0 \quad 0 \quad 0.25 \quad 0.25 \quad 0.5) \begin{pmatrix} 0.67 & 0.17 & 0 & 0 & 0.17 \\ 0.67 & 0 & 0.33 & 0 & 0 \\ 0 & 0.33 & 0.5 & 0.17 & 0 \\ 0 & 0 & 0.6 & 0.2 & 0.2 \\ 0 & 0 & 0 & 0.5 & 0.5 \end{pmatrix} \\ &= (\mathbf{0} \quad \mathbf{0.08} \quad \mathbf{0.28} \quad \mathbf{0.34} \quad \mathbf{0.3}) \end{aligned} \tag{20}$$

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

$$\begin{aligned}\text{Probability of } R_1 &= (0.75 \quad 0.25 \quad 0 \quad 0 \quad 0)\begin{pmatrix} 0.86 & 0.07 & 0.07 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0.4 & 0.4 & 0 & 0.2 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix} \\ &= (\mathbf{0.64} \quad \mathbf{0.05} \quad \mathbf{0.3} \quad \mathbf{0} \quad \mathbf{0}) \end{aligned} \tag{21}$$

4.3. *Results*

From the above results, the probability of a risk threat for R_5 during the following week is 0.3 at S_5, 0.34 at S_4, 0.28 at S_3, 0.08 at S_2, and 0 at S_1. This means that the portion of risk threat rating for R_5 will stay at around 64% (34% + 30%) of all potential defects during the next week. In the case of R_1, the values are set at 0 for S_5, 0 for S_4, 0.3 for S_3, 0.05 for S_2, and 0.64 for S_1, and 69% of all defects will stay at a risk threat state between S_1 and S_2.

In the early phases of a software project, many high-risk defects are typically reported with "Blocker, Critical" and "Always". After fixing the higher-risk defects, lower-risk defects should be found in accordance with the evolution of the software quality. In addition, we note that a high severity with a low frequency of defect occurrence still exists even at the end of the development process. In this paper, although the past data from each month are used, a more precise result can be obtained regardless of whether previous weekly or daily data are used. A larger number of states indicates a high resolution threshold, and more precise results can therefore be obtained. However, the complexity of the risk threat state and transition matrix increases depending on the increasing number of states. Therefore, it is necessary to define appropriate thresholds for each state in accordance with the applied domain and purpose. In addition, to obtain more precise results, the applied software project should be large in scale with a huge number of defects and active state transitions in short time periods, such as on a daily basis.

The model proposed in this paper has certain advantages. First, the proposed model quantifies the defect risk threat level as number of risks and their degree, which can be used as the criteria for judging the quality assurance. Second, the proposed model can be applied to diverse risk threats, from the lowest to the highest. Therefore, the risk threat can be analyzed independently or synthetically depending on the domain characteristics and their purposes. For example, the software release plan can be scheduled after examining the prediction results of a lower risk threat in the user software.

5. Related Works

A number of studies have investigated the risk prediction in software projects [13–15]. In particular, researches have concentrated on fault-proneness prediction

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

models. These models are focused on predicting which components of a newly developed software system will be most prone to failures. It has been noted that a system consists of components described by metrics such as the code complexity and number of pre-release changes. Some statistical models, such as a decision tree or logistic regression, are used based on these metrics to predict a component defect or system failure. According to their proposed method, the training of a statistical model for data regarding a similar software project, in which the metrics and fault proneness information have already been determined, is planned. This can be used as a model to predict a new system's proneness to faults. Moreover, for building a software risk estimation model, Gupta *et al.* [16] presented a quantitative model for risk control for a software project called SRAEM (Software Risk Assessment and Estimation Model).

Much of the recent industry research on software engineering has been conducted using the data mining of software repositories (MSRs), such as DTSs and version-control systems. For example, Hassan [17] introduced the concept of the entropy of changes, which is a measure of code change complexity. The entropy of changes was compared to the number of changes and previous bugs, and has often been found to provide better results. Kim *et al.* [18] proposed the change classification technique, which involves learning buggy change patterns from history, and predicting whether a new code change will lead to bugs. Zimmermann *et al.* [19] proposed a method for predicting the defect proneness of a file from the defect information extracted from the CVS/SVN repositories, and for predicting the defect proneness of a file. However, most of these studies have focused on a classification model to analyze how the code or behavior of a developer affects the software quality, and to predict whether the resulting code will be buggy or clean.

6. Limitations

Similar with other researches, our conclusions may be biased based on the particular data used to generate them. We have identified the following limitations of this study:

Human (tester) bias for evaluating the severity

A less-experienced tester engineer assigned the wrong severity level. Some defects were not actually critical from a requirement perspective, such as the UI or message display. However, a tester engineer marked them as a critical problem to encourage the developer to pay attention to their reports.

Non-platform based or small-scale software projects

As a case study, we intentionally chose a platform-based project with common parts. In addition, the number of defects must be high, and the defect transitions must be actively changed to ensure the accuracy of the prediction model. The validity of the transition probability matrix is difficult to guarantee for small software projects. The

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

proposed prediction model can work well when a sufficient amount of supporting data are available in an organization's software repository.

Not applicable to the case of a team change

The model we proposed only works if a single test team is involved. If the team members are changed, the severity and frequency of occurrence may differ from those determined by the team that initially built the prediction model. In the future, we need to conduct a sensitivity analysis against a test team change to evaluate the sensitivity of our prediction model.

7. Conclusions and Future Works

To perform better risk planning in the field of software engineering for avoiding, mitigating, and creating contingent responses against risks, identifying and predicting potential risk levels are important. In a software project, the quality of the developed software is the most crucial factor in software risk management.

In this paper, we proposed an RPM appropriate for evaluating software risk during management, and for predicting software risk from a quality perspective based on the repository history. Our method uses easily obtainable fields of a software defect tracking system such as Bugzilla, including the defect severity and frequency of occurrence as impact factors, as well as the probability of a risk matrix. During development, a large number of defects are reported to solve and track the software lifecycle, which are then logged in a repository. We map these attributes into a risk matrix for scoring their risks. Furthermore, a risk matrix is used to applying an MC as a prediction model. The MC model used in our study has been used widely, and has been proven effective for various types of information processing. We evaluated the proposed approach using data mined from the defect records of practical mobile software projects actually developed in the industry. The experimental results demonstrate that our approach is effective to quantize and estimate software risk threat levels.

We believe that our model can help project managers better understand and improve the management activity of software development efforts, such as anticipating problems under uncertainty, thereby controlling the resources more effectively, gaining insight into the likely quality to be achieved from the development efforts, and ensuring the success of the development objectives, such as the time-to-market and the software quality.

In future, we will focus on the following topics and explore whether the results obtained in this paper are effective for software project management. We are currently working on evaluating the effectiveness of the proposed model wherein metrics can be selected and can improve the software project management. Furthermore, statistical inferences in the repository data, leading to systematic bias which threatens the precision of prediction model, were investigated [20]. We plan to apply interference elimination to improve the precision of the proposed model.

Int. J. Soft. Eng. Knowl. Eng. 2015.25:1017-1032. Downloaded from www.worldscientific.com
by PONTIFICIA UNIVERSIDADE CATOLICA DO PARANA on 04/20/26. Re-use and distribution is strictly not permitted, except for Open Access articles.

Acknowledgments

This research was supported by the Next-Generation Information Computing Development Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Science, ICT & Future Planning (2012M3C4A7033345), and Basic Science Research Program through the National Research Foundation of Korea(NRF) funded by the Ministry of Education, Science and Technology (2012R1A1A2009021).

References

1. Research and Markets, Software: Global Industry Guide, 2013.
2. The Standish Group, CHAOS MANIFESTO, 2013.
3. IEEE Std 1058–1998, IEEE Standard for Software Project Management Plans, 1998.
4. The Standish Group, EXTREME CHAOS, 2001.
5. Project Management Institute (PMI), *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 4th ed. (Project Management Institute, 2008).
6. CMMI Product Team, Capability Maturity Model Integration (CMMI SM) Version 1.1, CMMI for Systems Engineering, Software Engineering, Integrated Product and Process Development, and Supplier Sourcing (CMMI-SE/SW/IPPD/SS, V1.1), 2002.
7. P. Tianyin, Development of software project risk management model review, *in 2nd International Conference on Artificial Intelligence, Management Science and Electronic Commerce*, 2011, pp. 2979–2982.
8. L. Wallace, M. Keil and A. Rai, Software project risks and their effect on outcomes, *Commun. ACM* **47**(4) (2004) 68–73.
9. J. Jiang and G. Klein, Software development risks to project effectiveness, *Journal of Systems and Software* **52**(1) (2000) 3–10.
10. Bugzilla, available at: http://www.bugzilla.org/ last visited 2013., Git, available at: http://git-scm.com/ last visited 2013, Jira, available at: http://jira.dspace.org last visited 2013.
11. R. Paul, P. R. Garvey and Z. F. Lansdowne, Risk matrix: An approach for identifying, assessing, and ranking program risks, *Air Force Journal of Logistics* **25**(1) (1998) 16–19.
12. W. K. Ching and M. K. Ng, *Markov Chain: Models, Algorithms and Applications* (Springer Science, 2006).
13. T. Menzies, J. Greenwald and A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Trans. Software Eng.* **33**(1) (2007) 2–13.
14. N. Nagappan, T. Ball and A. Zeller, Mining metrics to predict component failures, in *Proc. 28th Int. Conf. Software Eng.*, 2006, pp. 452–461.
15. N. Nagappan and T. Ball, Use of relative code churn measures to predict system defect density, in *Proc. 27th Int. Conf. Software Eng.*, 2005, pp. 284–292.
16. D. Gupta and M. Sadiq, Software risk assessment and estimation model, in *Int. Conf. Computer Science and Information Technology*, 2008, pp. 963–967.
17. A. E. Hassan, Predicting faults using the complexity of code changes, in *Proc. 31st Int. Conf. Software Eng.*, 2009.
18. S. Kim, E. Whitehead Jr. and Y. Zhang, Classifying software changes: Clean or buggy? *IEEE Trans. Software Engineering* **34** (2008) 181–196.
19. T. Zimmermann, R. Premraj and A. Zeller, Predicting defects for eclipse, predictor models in software engineering, in *PROMISE'07: ICSE Workshops*, 2007.
20. C. Bird, A. Bachmann, E. Aune, J. Duffy, A. Bernstein, V. Filkov and P. Devanbu, Fair and balanced?: Bias in bugfix datasets, in *Proc. ESEC/FSE '09*, 2009, pp. 121–130.